

Philips Biosensing Platform

SY1903 PSP-S API Specification

Rev. 1.2 — 07-Aug-2023

Approved

Philips Confidential

Document information

Info	Content
Keywords	Biosensing Platform, API, Sleep, Specification
Abstract	This document describes the application programming interface for PSP-S library.

PHILIPS

Revision history

Rev	Date	Author	Description
1.0	12-Aug-2021	JaWo	First Release
1.1	25-Nov-2021	JaWo	Correct typo
	24-May-2022 07-Nov-2022	JaWo JaWo	Update Legal Information Update legal disclaimer, update date format
1.2	07-Aug-2023	JaWo	Update version number and reference version number Amend document title Update latency

CoreKinect LLC

Table of Contents

1	Introduction	4
1.1	Purpose & scope	4
1.1.1	PSP-S variants	4
1.1.2	PSP-S versions	4
1.2	Important safety notice	4
1.3	Definitions, acronyms & abbreviations	4
1.4	References	5
2	Application context	6
3	Functions	7
3.1	Life cycle; single-user versus multi-user	7
3.2	Extraction	8
3.3	Calling protocol	8
3.4	Concurrency limitations	9
4	Metrics	10
4.1	Input metrics	10
4.2	Output metrics	10
4.3	Sleep Session	10
4.4	Private Data corruption robustness	12
4.5	Private Data source time adjustments	12
5	Deployment formats	13
5.1	C library for Linux	13
5.1.1	Lifecycle control	13
5.1.2	Code architecture	13

1 Introduction

1.1 Purpose & scope

This specification describes the Philips Biosensing Platform – Sleep Sensing by PPG software library (referred to hereafter as ‘PSP-SL’). The main functionality of PSP-S library is to extract health related metrics of a person. PSP-SL comes in different deployment formats, suited for specific computational platforms.

The purpose of this specification is to give a high-level overview of the PSP-SL functionality, mechanisms and metrics. More precisely, to describe the requirements and protocol that an application (typically, software running on a mobile device) must follow to use PSP-SL.

The detailed software API aspects (such as function signatures and datatypes) are beyond the scope of this specification; they are provided in documentation accompanying the various deployment formats (refer to section 5).

1.1.1 PSP-S variants

The contents of this specification apply to all products, being variants of the PSP-S library, except where explicitly stated. In particular, the set of supported extracted metrics differs per variant.

1.1.2 PSP-S versions

This specification is generally applicable to all PSP-S versions 1.4 or above except where explicitly stated.

1.2 Important safety notice

The accuracy of the metrics, output by PSP-SL, may vary due to a variety of causes. The PSP-S application shall at all times and by all necessary means warrant the user’s safety also in occurrence of metric inaccuracies.

In particular, using the sensing technology that eventually results in the input data for PSP-SL outside of its specified operational conditions increases the probability of inaccuracies. While this sensing technology is out of scope for PSP-S, it is still strongly advised to limit the application’s intended use within the sensing technology’s operational conditions.

1.3 Definitions, acronyms & abbreviations

Abbreviation	Description
ACC	Acceleration
API	Application Programming Interface
BPM	Beats Per Minute
PSP-S	Philips Biosensing Platform – Sleep Sensing by PPG, a metric extraction co-processing library running on mobile, used together with PSP.
PSP-SL	Philips Biosensing Platform – Sleep Sensing by PPG library (PSP-S library)

HW	Hardware
NTP	Network Time Protocol
PPG	Photo plethysmography measures the volumetric changes of the arteries caused by the pumping of the heart. From this measurement average heart rate and heart rate variability can be extracted.
PSP	Philips Biosensing Platform metrics extraction library

1.4 References

Abbrev.	Title	Author(s)	Version
SY1901	SY1901 Philips PSP API Specification	Philips	1.3
SY1902	SY1902 Philips PSP Metrics Specification	Philips	1.3

CoreKinect LLC

2 Application context

The main function of PSP-S is to extract health related metrics of a person whose extraction is resource (computing or memory) intensive. And hence PSP-S library should run in a resource rich environment, such as on a mobile device.

Figure 1 shows how PSP-S is applied with the PSP software library.

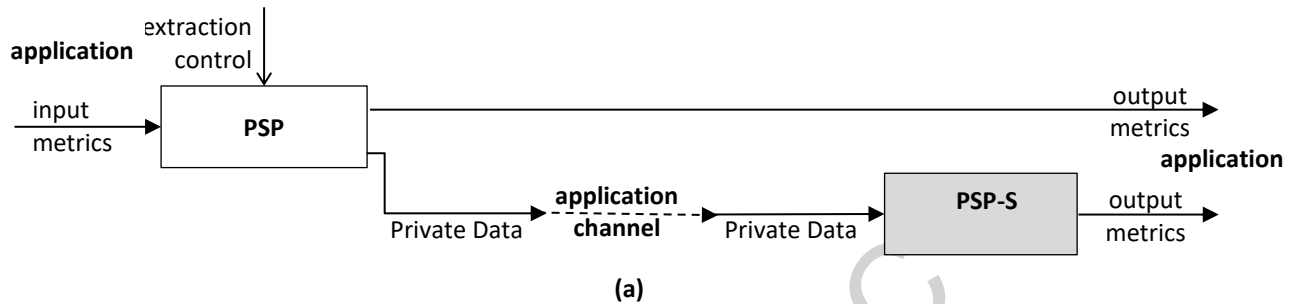


Figure 1 – PSP-S applied with PSP library

Typically, PSP library is contained in a wearable device, and extract health related output metrics of a person out of a combination of input metrics, in particular sensor metrics data (such as acceleration) and external personal metrics (e.g. the person's height). These output metrics are thus available on the wearable device, and can e.g. be used for the user interface on the wearable device.

In addition, PSP library provides the Private Data metric at a rate of 1 packet per minute. It is the responsibility of the application (typically comprising the wearable host processor, connectivity such as Bluetooth, and a mobile app incorporating PSP-S) to transfer these packets to PSP-S integrally and in the **exact same order** as it was provided. PSP-S will then extract its output metrics.

The application has the possibility to enable or disable output metrics via the extraction control of PSP. This enabling or disabling may be performed at any moment in time, and for all output metrics individually. The extraction control governs not only the output metrics of PSP library, but also the output metrics of PSP-S. In other words, PSP-S has no extraction control of its own; output metrics of PSP-S have to be enabled or disabled via PSP. Refer to [SY1901] for extraction control details.

PSP-S library provides time information along with its output metrics. For this, PSP library internally keeps track of elapsing Time, but the absolute time has to be set (at least once) by their respective application. See [SY1901] for more details.

3 Functions

PSP-S supports the following functions.

Function	Semantics
lifecycle	
<instantiate>	Create an PSP-S instance. How instances are created depends on the actual deployment format; refer to section 5 for details per deployment format
<terminate>	Terminate the specified PSP-S instance. How instances are terminated depends on the actual deployment format; refer to section 5 for details per deployment format
extraction	
ListRequiredMetrics	Get a list of all input metrics that are required for extracting the output metrics.
ListUpdatedMetrics	Get a list of all output metrics that were updated as a result of Process
SetMetric	Set the value of the specified input metric
GetMetric	Get the value of the specified output metric
Process	Perform the actual extraction, based on enabled output metrics and on the input metrics having been set since the last Process.
Configuration	
GetVersion	Get the version of PSP-S, uniquely identified by the combination of returned elements • main version • major revision • minor revision • internal configuration (not interpretable by the application, intended only for support purposes)

Table 1 – PSP-S functions

3.1 Life cycle; single-user versus multi-user

Before starting the metrics extraction functionality, an instance of PSP-S must be created. Likewise, the PSP-S instance should be terminated after definitively ending the extraction operation.

The PSP-S instance maintains the state of the PSP-S extraction, which actually is the result of all input metrics data that PSP-S has processed so far. Naturally, these input metric data are person-specific. Therefore, an instance of PSP-S is specific for (i.e. associated with) *a single person*.

In conjunction with a typical single-user device such as an activity tracker, sportswatch or smartwatch, just one PSP-S instance needs to be used. This also means that the PSP-S instance is *person specific* and may not be used alternately (i.e. shared over time) by multiple persons.

In case a device does need to be alternately used by multiple persons, or a computing device running PSP-S should support multiple devices (and thus their multiple users), this can be realized by multiple PSP-S instances (one per device user) which can be maintained independently in parallel.

3.2 Extraction

While PSP-S is extracting, `ListRequiredMetrics()` indicates which input metrics are required in relation to the enabled output metrics. It is the responsibility of the application to constantly provide any required input metrics and to call `Process()` so that the extraction of output metrics can take place (see also the protocol in section 3.3).

After each `Process()`, PSP-S indicates via `ListUpdatedMetrics()` which metrics have been updated, so that the application may selectively get these via `GetMetric()`. It is not mandatory to get updated metrics; however, the metric value may be updated (i.e. overwritten) in a subsequent `Process()`.

The application is not bound to getting only updated metrics; it may get any metric at any time, yielding the last updated value. As long as the value is not updated, multiple gets will result in the same metric value being repeatedly provided¹.

The list of required input metrics may actually vary dynamically over time. By this, PSP-S allows the application to minimize resource usage.

3.3 Calling protocol

PSP-S functions may be called according to the protocol state machine of Figure 2. Note the states indicated are conceptual only, PSP library does not report its state at the API.

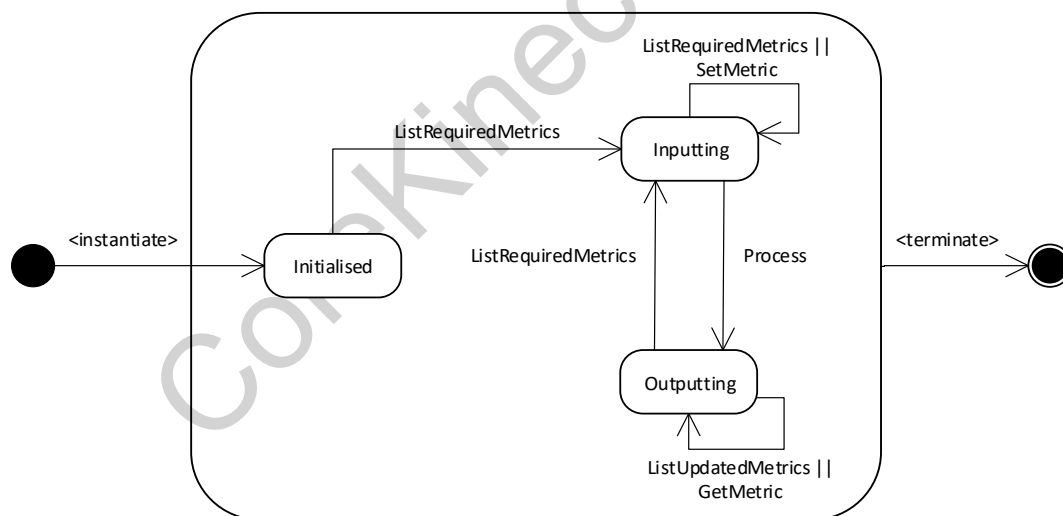


Figure 2 – Protocol state machine

The functions specified in section 3 may be called only in specific states, as listed in Table 2.

¹ The application can also detect this from the index field of the metric being unchanged; refer to section 4.

Function	State		
	Initialised	Inputting	Outputting
<initiate>			
ListRequiredMetrics	x	x	x
SetMetric		x	
Process		x	
ListUpdatedMetrics			x
GetMetric			x
GetVersion	x	x	x
<terminate>	x	x	x

Table 2 – Functions and protocol states: ‘x’ denotes that function may be called

The example pseudo-code in Figure 3 adheres to the protocol. The functions are called in a simple, repetitive fashion.

```

“initiate PSP-S”           // deployment format specific
while ( “extraction required” ) // ‘forever’ in many applications
  ListRequiredMetrics ( &List )
  for ( each metricID in List ) // Private Data, ...
    “gather metricData”     // only one value per metric at a time
    SetMetric(metricID, metricData )
  end for
  Process ( )              // let PSP-S process anything it can
  ListUpdatedMetrics ( &List )
  for ( each metricID in List )
    GetMetric( metricID, &metricData )
    “interpret metricData”
  end for
end while
“terminate PSP-S”         // deployment format specific

```

Figure 3 – Example API calling

Note that in state Inputting, 1 value per required input metric must be set before calling Process(). This implies also that the time jumps between subsequent Process calls is determined by the smallest update interval of all required input metrics, as specified in [SY1902]. E.g. for Private Data, the interval is 60 seconds.

3.4 Concurrency limitations

PSP-S is designed to be used synchronously within an PSP-S instance: the application **may not make** concurrent function calls for **one PSP-S instance**. In other words, all functions for that instance must be called one-by-one sequentially, in the order of the protocol specified in section 3.3.

The application **is allowed** to make concurrent PSP-S calls for **different PSP-S instances** (e.g. in a multi-user application as described in section 3.1).

4 Metrics

As described before, PSP-S requires & accepts certain metrics as input and from these, extracts output metrics. All metrics comprise:

- an ID, uniquely identifying the type of metric;
- an Index (range 0..255), which is incremented at every metric value update, and enabling the application to discern consecutive values (e.g. for polling purposes);
- a quality indicator (range 0-4), which indicates how reliable the metric value is: from 0 = fully unreliable to 4 = maximally reliable;
- the actual metric value itself.

Detailed syntax and semantics of all metrics are given in [SY1902].

4.1 Input metrics

PSP-S accepts input metrics as listed in Table 3. Which of these input metrics are actually required for PSP-S to offer its functionality depends on which output metrics are supported, i.e. on the PSP-S variant. Detailed syntax and semantics are given in [SY1902]. Before providing the input metrics, Private Data, Sleep Stages metrics should be enabled in the PSP library.

Metric	Notes
Private Data	See also section 2

Table 3 – PSP-S input metrics

4.2 Output metrics

PSP-S provides output metrics as listed in Table 4. Which of this list of output metrics is actually supported or not depends on the PSP-S variant.

Metric	Notes	Input metrics required to be set
Sleep Stages	See also sections 4.3, 4.4 and 4.5	Private Data

Table 4 – PSP-S output metrics

4.3 Sleep Session

The sleep patterns of a user are described via the concept of Sleep Sessions. A Sleep Session describes a single session (typically, a night) of sleep of the user and contains the following information:

- Session Start Time and Session Start Source;
- Session End Time and Session End Source;
- A sequence of sleep stages;
- Sleeping Heart Rate.

The start time of a sleep session is either determined automatically by analysis of the user's motion patterns, or triggered by setting the Sleep Preference input metric of PSP [SY1901]. A Sleep Session is started automatically when the user starts a period of physical inactivity and remains inactive for at least 20 minutes. An automatically detected start of a

Sleep Session will be overruled by setting Sleep Preference to “Intent to Sleep”. The Session Start Source indicates whether the session was started automatically or by using Sleep Preference.

The end of a Sleep Session is determined from a combination of the user’s motion patterns, and setting the Sleep Preference input metric of PSP library to “No Intent to Sleep”. In this way a Sleep Session is ended even if the user forgets to set Sleep Preference after waking up.

- A Sleep Session is ended manually if the Sleep Preference input is set to “No Intent to Sleep.”
- A Sleep Session is ended automatically when the user starts a period of physical activity and remains active for at least 60 minutes.
- The setting of the Sleep Preference input gets priority over automatic ending due to physical activity, unless the setting of Sleep Preference takes place at least 20 minutes later than the actual start of physical activity. In that case the sleep session is ended automatically, and the Sleep Preference setting is ignored.

When Sleep Preference (Intent to Sleep) is set, it gives a trigger to the system to enforce a start or stop session at that particular moment. If one is enforced to set Sleep Preference (after enabling Sleep Stages/Private Data) then “unspecified” can be used.

In addition, the system takes into account only 1 Sleep Preference setting per minute, setting it more will tend to discard previous values of that minute.

The Session Start Source and Session End Source fields are relevant for the application if it calculates derived sleep metrics like Sleep Onset Latency from the Sleep Session.

The Session End Source indicates whether the session was ended automatically or by using Sleep Preference or End of Stream (special case explained below).

A special case occurs when the Sleep Session is interrupted by disabling of the Sleep Stages metric on PSP-S. Normally this interruption is only seen at the end of the night but also can be triggered by unintended disabling of the Sleep Stages metric (May be due to false off skin detection implemented over the night which turns off all metrics for power saving). In this case it is not clear whether the Sleep Session has ended, or may be resumed. Therefore, in case of interruption, a Sleep Session with a pending end (End of Stream) is output, indicated by the Session End Source (refer to Sleep Stages metric description in [SY1902]). After the re-enabling of the Sleep Session metric on PSP, and the arrival of sufficient new Private Data in PSP-S, a definitive Sleep Session is output. If Sleep Session End was ended by End of Stream, the PSP-S library retains all previous Private Data and waits if logging will resume. If it resumes within a certain period of time, then the Sleep Session will be continued with all previous Private Data. Otherwise, all internal data will be flushed.

The minimum length of a Sleep Session is 20 minutes; the maximum length equals 960 minutes. In case the end of a Sleep Session is determined automatically, the system will return the sleep session with an algorithmic latency of 100 minutes after the actual session end time. A Sleep Session ended by using Sleep Preference is returned within 1 minute after the actual session end time. A temporary Sleep Session ended by disabling of the Sleep Metric and Private Data Metric is returned within a few seconds , sooner than that if ended by Sleep Preference setting.

For each 30s time interval (or epoch) between Session Start Time and Session End Time, the Sleep Session reports a detected Sleep Stage value from the set {Wake, REM sleep, Light sleep, Deep sleep, Unspecified}. The Unspecified value indicates that it was not possible to determine a Sleep Stage value for this epoch. This may occur in case of interruptions (see above), and in case of low quality PPG data at the PSP library input (e.g. due to poor skin contact of the originating PPG sensor).

Sleeping Heart Rate is calculated as the minimum of a sequence of Heart Rate values obtained during the entire sleep session. Each Heart Rate value is calculated as an average heart rate over a time window of 4.5 minutes length.

4.4 Private Data corruption robustness

PSP-S requires that all Private Data packets arrive in strictly original chronological order, exactly as they were output by PSP library, and without duplications of packets.

When receiving incorrectly sorted Private Data packets, PSP-S will return an exception and ignore the inordinate Private Data.

PSP-S is robust to incidental loss of Private Data, though this may impact the output metric values. The precise effect of data corruption on the PSP-S output metrics is metric specific (Table 5).

Metric	Private Data corruption effects
Sleep Stages	Time intervals during a Sleep Session of less than 60 minutes for which Private Data packets are missing result in 'Unspecified' sleep stages. Time intervals of missing packets beyond 60 minutes will lead to missing or prematurely ended Sleep Sessions.

Table 5 – Private Data corruption effects on output metrics

4.5 Private Data source time adjustments

The Private Data packets contain embedded time related information, inserted by their source i.e. PSP library. This time information is used to relate the extracted metrics to the original time at which the corresponding Private Data was generated by PSP library.

As explained in [SY1901], this time may drift from the true time. The application may therefore adjust the time according to a time reference such as an NTP server by setting the Time input metric of PSP library. The effects of such Time adjustments on the output metrics of PSP-S are metric specific (Table 6).

Metric	Time adjustment effects
Sleep Stages	Adjustments of Time of less than 60 seconds will not affect Sleep Sessions output except for a corresponding adjustment of the Session Start Time and/or Session End Time. Larger time adjustments may have a more significant influence. Especially, a backward time adjustment of more than 60 seconds is interpreted as a packet sorting error. An error code is returned, and an ongoing Sleep Session may be lost.

Table 6 – Time adjustment effects on output metrics

5 Deployment formats

By design, PSP-S targets a variety of execution platforms, now and in the future. The actual ways to call PSP-S are platform and programming language specific. The available deployment formats are listed below.

5.1 C library for Linux

This deployment format provides the following deployment format specific functions.

Function	Semantics
lifecycle control	
GetDefaultParams	Get the default instance parameters before creating an instance, comprising minimum size of the instance memory buffer that the application must provide to Initialise
Initialise	Initialise and claim the provided instance memory, so as to create a new instance
Terminate	Terminate the specified instance and release the instance buffer memory.

Table 7 – C library image specific functions

5.1.1 Lifecycle control

In this deployment format, memory for the instance must be provided by the application, i.e. PSP-S does not allocate memory on its own. The application creates an instance by allocating memory of the minimum size indicated by GetDefaultParams() or bigger, and passing it to Initialise().

During the entire PSP-S lifecycle, the application maintains and does not modify the instance memory. The application passes the instance with any PSP-S function calls.

At the end of the PSP-S lifecycle, i.e. at the definitive end of the extraction, Terminate() should be called.

In a multi-instance setting (such as for a device shared by multiple users), the application could keep multiple PSP-S instances (one per user) in memory. Alternatively, the application could e.g. swap PSP-S instance contents between memory and any persistent storage in between calls to PSP-S.

5.1.2 Code architecture

In this deployment format, the software is supplied as a pre-built binary library. The software consists of several files (Table 8).

File	Contents
psp_s.h	PSP-S functions & definitions
fx_datatypes.h	datatype definitions
psp_s.a	binary library

Table 8 - C library code architecture

Legal information

Disclaimers

This document may be subject to change without notice. Philips Electronics Nederland B.V. does not give any representations or warranties as to the accuracy or completeness of information included herein and shall have no liability for the consequences of use of such information.

This document is provided “as is” with no warranties, expressed or implied, including but not limited to any implied warranty of merchantability, fitness for a particular purpose, or freedom from infringement. Koninklijke Philips N.V. or its Affiliates may have patents or pending patent applications, or other intellectual property rights that relate to the described subject matter. Philips and its logo are trademarks of Koninklijke Philips N.V. All other names are the trademark or registered trademarks of their respective holders. The furnishing of this document does not provide any license, expressed or implied, by estoppels or otherwise, to any such patents, trademarks or copyrights, or other intellectual property rights. Philips Electronics Nederland B.V. assumes no responsibility for errors or omissions in this document; nor does Philips Electronics Nederland B.V. make any

commitment to update the information contained herein. This document is subject to change without notice.

Patents

Notice is herewith given that the subject device uses one or more of the following patents and that each of these patents may have corresponding patents in other jurisdictions.

Philips has been awarded a number of patents on the technology used in the Philips Biosensing Platform offering in Europe, Asia and the United States.

Trademarks

Notice: All referenced brands, product names, service names and trademarks are property of their respective owners.

CoreKinect LLC